

Tarea7: ¿Cómo garantizo que los tiempos sean reales?

Angel Alam Mateo Encarnacion 2024-0686

Quería hacer este documento para explicar un poco la lógica que armé detrás del código. Mucha gente piensa que medir los tiempos por software puede ser impreciso, pero aquí quiero demostrar por qué es físicamente imposible que un humano con un cronómetro le gane en exactitud a este sistema [cite: 1].

El cronómetro

En lugar de usar contadores raros que sumen números vuelta por vuelta, decidí ir directo al grano y usar el hardware de la placa. La clave de todo es `esp_timer_get_time()` [cite: 1]. Esta función no es un invento mío, es un reloj interno del ESP32 que cuenta los microsegundos de forma independiente desde que conectas la placa [cite: 1].

Entonces, cuando quiero saber la reacción de un jugador (como en el primer juego), simplemente miro el reloj cuando enciendo el LED y lo vuelvo a mirar cuando el jugador presiona el botón. La resta de esos dos momentos me da el tiempo real [cite: 1]. Nuestro código da vueltas cada 5 milisegundos, así que ese es nuestro único y diminuto margen de error [cite: 1].

Filtrando trampas y toques accidentales

Un dolor de cabeza común es cuando alguien deja el dedo pegado en el botón y el sistema se vuelve loco contando cientos de toques. Para arreglar eso, agrupé un par de soluciones:

No más dobles toques: Uso unas variables (como `boton1_anterior`) para recordar cómo estaba el botón hace 5 milisegundos [cite: 1]. El sistema solo cuenta el toque si ve que el botón pasó de estar suelto a presionado en ese instante exacto (esto se llama detección de flanco) [cite: 1]. Si lo dejas pegado, no pasa nada.

Obligando a alternar: En el Juego 2, donde tienes que tocar un botón y luego el otro, uso una memoria simple llamada `ultimo_boton` [cite: 1]. Si te pones a machacar el Botón 1 a lo loco, el sistema te ignora. Solo suma un paso cuando realmente pasas al Botón 2 y viceversa [cite: 1].

Para el juego 2 los promedios sin enviar datos basura

Cuando calculo el promedio de la segunda reacción, no uso fórmulas mágicas ni estimaciones al aire. Lo que armé fue un búfer circular [cite: 1]. Básicamente es un arreglo llamado `tiempos_reaccion2` que tiene exactamente 10 espacios [cite: 1].

El sistema va llenando esos espacios con jugadas reales, y si llega al final, empieza a sobrescribir desde el principio [cite: 1]. Pero lo mejor de esto es que bloqueé el envío de datos por MQTT: hasta que una variable llamada `cantidad_reacciones2` no me confirme que tenemos las 10 jugadas completitas y válidas, no se publica nada [cite: 1]. Cero datos a medias.

La máquina de estados

En vez de hacer una tabla aburrida (como intenté antes), prefiero explicarles cómo se organiza todo. Imaginen que el sistema tiene "modos" (`ESPERANDO_LED`, `JUEGO_TERMINADO`, etc.) [cite: 1].

- Si alguien presiona un botón cuando el LED aún no enciende, el sistema sabe que está en estado `ESPERANDO_LED` y automáticamente lo marca como una falsa salida, en vez de registrar un tiempo negativo o absurdo [cite: 1].
- Tengo una bandera llamada `boton1_suelto` que funciona en equipo con los estados para asegurarse de que la primera reacción ya pasó y no se dupliquen eventos en la misma ronda [cite: 1].
- Cuando termina el juego (`JUEGO_TERMINADO`), obligo a que la persona suelte todos los botones antes de poder empezar de nuevo, limpiando cualquier error [cite: 1].

Para resumir

Con todo esto operando junto, los números no mienten [cite: 1]. El hardware se encarga del tiempo puro, la detección de flancos se encarga de los dedazos, y la máquina de estados pone las reglas [cite: 1]. Es un sistema cerrado y a prueba de mañas, midiendo exactamente lo que el jugador hace en la vida real [cite: 1].